Livrable 3 — Démarche éco-responsable

SmartTicket — Justification environnementale des choix techniques

---

3.1 Référentiels appliqués

Les choix techniques de SmartTicket sont évalués au regard de trois référentiels :

- **RGESN** (Référentiel Général d'Écoconception de Services Numériques) — ARCEP/ARCOM 2024 Définit 79 critères organisés en 8 thématiques : stratégie, spécifications, architecture, UX/UI, contenus, front-end, back-end, hébergement.
- **GR491** (Guide de Référence de Conception Responsable de Services Numériques) — INR 2023 491 pratiques couvrant tout le cycle de vie d'un service numérique, du besoin à la fin de vie.
- **Green IT / Éco-index** Métriques orientées performance web et réduction de l'empreinte côté client.

3.2 Choix techniques justifiés sous l'angle éco-responsable

Choix technique réel (lu dans le code)	Justification éco-responsable	Référentiel
Mistral AI API (mistral-small-latest, mistral-embed)	Mistral AI est une entreprise française. Ses datacenters sont hébergés en Europe avec un mix énergétique à faible intensité carbone (France : ~70 g CO_2/kWh vs 400-500 g/kWh au charbon). Le choix de `mistral-small-latest` plutôt qu'un modèle plus grand (ex: GPT-4) réduit la consommation GPU par requête.	RGESN 8.x Hébergement
pgvector intégré à PostgreSQL (vs base vectorielle dédiée Pinecone/Weaviate)	Mutualisation de l'infrastructure : un seul service PostgreSQL gère à la fois les données relationnelles et les vecteurs. Élimine un service supplémentaire, réduit le trafic réseau inter-services et la consommation en RAM/CPU.	GR491 — Réduire le nombre de services
Streaming SSE (`StreamingResponse`, `text/plain`) sur `/v1/ask/stream`	Évite de bufferiser toute la réponse LLM en mémoire avant envoi. La réponse est transmise token par token, réduisant la latence perçue et la mémoire allouée côté serveur. Pas de polling — connexion unique jusqu'à la fin du stream.	RGESN 7.x Back-end
Conteneurs Alpine (`redis:7-alpine`)	Image Redis Alpine ~30 Mo vs ~120 Mo pour la version Debian. Moins de couches Docker = moins de stockage, moins de surface d'attaque, démarrage plus rapide.	RGESN 8.x Hébergement
python:3.11-slim (backend)	Image slim (~120 Mo) vs image complète (~900 Mo). Réduit la taille des artefacts Docker et la durée des	RGESN 8.x Hébergement

Choix technique réel (lu dans le code)	Justification éco-responsable	Référentiel
	builds CI.	
node:20-slim (frontend)	Image slim Node.js. Build en deux phases implicites (install + build + démarrage production).	RGESN 8.x Hébergement
Cache Redis prévu (non encore connecté)	L'architecture prévoit Redis pour mettre en cache les résultats d'embeddings et de génération, évitant des appels redondants à l'API Mistral (facturation à l'usage + empreinte carbone).	RGESN 7.x Back-end
Pagination implicite sur les analytics (`?days=N`)	Le paramètre `days` sur `/v1/analytics/stats` limite la fenêtre temporelle des données agrégées. Évite de charger l'ensemble de la base pour des requêtes analytiques.	GR491 — Réduire les transferts de données
Chunking des transcripts (1000 chars, overlap 150)	Les transcripts de sessions sont découpés en petits chunks avant vectorisation (`TRANSCRIPT_CHUNK_SIZE=1000`). Évite d'envoyer de longs textes d'un coup à l'API Mistral, réduisant les tokens consommés par requête d'embedding.	RGESN 7.x Back-end
Résumé IA à la clôture de session	Plutôt que de vectoriser l'intégralité du transcript à chaque recherche RAG, un résumé compact (5-8 lignes) est généré une fois et indexé. Réduit le nombre de tokens traités par Mistral pour les recherches futures.	GR491 — Optimiser les algorithmes
Recharts (SVG, pas WebGL)	La bibliothèque graphique utilise SVG natif, pas WebGL. SVG est plus léger en GPU, accessible, indexable.	RGESN 5.x Contenus
Lucide React (icônes SVG inline)	Icônes SVG tree-shakable (seules les icônes importées sont incluses dans le bundle). Pas de sprite sheet ni police d'icônes complète chargée.	RGESN 6.x Front-end
Shadcn/UI (composants copiés, pas une dépendance npm)	Les composants sont copiés dans le projet, ce qui permet le tree-shaking au niveau du bundle. Pas de bibliothèque UI complète chargée.	RGESN 6.x Front-end
Background tasks pour l'ingestion	Les opérations d'ingestion (scraping, embedding batch) sont exécutées en arrière-plan sans bloquer la connexion HTTP. Réduit le nombre de connexions ouvertes simultanément.	RGESN 7.x Back-end
Bcrypt (facteur de coût adapté)	Hachage bcrypt avec `gen_salt('bf')` — facteur de coût adaptatif. Plus sécurisé qu'argon2id mais aussi plus intensif en CPU. Acceptable pour un service à faible charge.	GR491 — Sécurité proportionnée

3.3 Métriques d'éco-conception mesurables

Ces métriques sont à mesurer sur l'URL de pré-production une fois confirmée.

Métriques cibles par page

Page	Outil	Métrique	Cible
`/login`	Lighthouse	Performance score	≥ 85
`/` (dashboard)	Lighthouse	Performance score	≥ 85
`/ai-assistant/[id]`	Lighthouse	Performance score	≥ 80
Toutes	Ecoindex.fr	Score éco-index	≥ C (≥ 50/100)
Toutes	DevTools Network	Poids total de la page	< 1 Mo
Toutes	DevTools Network	Nombre de requêtes HTTP	< 30

Empreinte carbone estimée par conversation

Méthodologie Boavizta / Green Algorithms :

- 1 question RAG ≈ 1 appel `mistral-embed` (~500 tokens input) + 1 appel `mistral-small-latest` (~800 tokens input + ~200 tokens output)
- Estimation : ~0,002 kWh/conversation (modèle small, inférence cloud UE)
- Equivalent CO_2 : ~0,14 g CO_2eq/conversation (mix électrique français 70 g/kWh)

À mesurer précisément via l'API Mistral (compteur de tokens dans les réponses) et le calculateur Boavizta.

Métriques de performance API

Endpoint	Métrique	Cible
POST /v1/ask/stream	Temps au premier token (TTFT)	< 800 ms
POST /v1/login	Latence P95	< 300 ms
GET /v1/analytics/stats	Latence P95	< 500 ms
POST /v1/knowledge-base/ingest-url	Durée job background	< 60 s pour < 10 URLs

---

3.4 Hébergement

Hébergeur actuel : Render.com (pré-production)

Critère	Valeur connue
Localisation datacenter	US-East (Oregon) par défaut sur le plan Free
Certifications	SOC 2 Type II, ISO 27001
Mix énergétique	AWS us-east-1 : ~28% renouvelable (source AWS Sustainability Report 2023)

Critère	Valeur connue
PUE	~1.2 (datacenters AWS Oregon)
Engagement net zéro	Render s'engage sur la neutralité carbone via AWS

Écart et recommandation

Le plan Free de Render place les services sur des régions US par défaut. Pour une meilleure empreinte carbone :

Option	Bénéfice éco	Complexité
Scaleway (Paris)	Datacenters en France, mix électrique < 70 g CO_2/kWh, ISO 50001	Migration Dockerfile → Scaleway Container Registry
OVHcloud (Roubaix)	Refroidissement adiabatique, PUE < 1.3, mix FR	Configuration DNS + deploy
Fly.io (region fra)	Région Frankfurt disponible, proche mix énergétique européen	`fly.toml` à créer

Recommandation : migrer vers Scaleway ou OVHcloud pour la production afin d'aligner l'hébergement avec les valeurs déclarées (mix énergétique bas carbone, souveraineté des données).

3.5 Bonnes pratiques éco-conception constatées dans le code

Pratique	Statut	Preuve dans le code
Streaming SSE (pas de polling)	✅ Implémentée	`backend/routers/ai.py:StreamingResponse` + `frontend/app/api/ask/route.ts`
Tree-shaking des icônes	✅ Implémentée	Imports unitaires Lucide (`import { X } from "lucide-react"`) dans les composants
Composants UI tree-shakable	✅ Implémentée	shadcn/ui copiés localement dans `frontend/components/ui/`
Requêtes SQL agrégées	✅ Implémentée	`analytics.py` utilise `func.count`, `func.date_trunc` (agrégation SQL, pas Python)
Chunking des documents	✅ Implémentée	`dependencies.py:chunk_text()` + `TRANSCRIPT_CHUNK_SIZE=1000`
Background tasks (non-bloquant)	✅ Implémentée	`knowledge.py:BackgroundTasks` pour l'ingestion
Retry avec backoff exponentiel	✅ Implémentée	`mistral_client.py:_request_with_retry()` — évite les appels en rafale

Pratique	Statut	Preuve dans le code
Variables d'env configurables	✅ Implémentée	`KB_TOP_K`, `KB_MAX_CONTEXT_CHARS` permettent d'ajuster la consommation de tokens
Image Docker slim/alpine	✅ Implémentée	`python:3.11-slim`, `node:20-slim`, `redis:7-alpine`
Conteneur backend sans dépendances inutiles	⚠️ Partielle	`requirements.txt` contient `chromadb` et `ollama` (legacy, non utilisés en prod) — à nettoyer
Lazy loading des routes	⚠️ Partielle	Next.js App Router charge les pages à la demande par défaut, mais pas de `React.lazy` explicite sur les composants lourds (recharts)
Compression Brotli/Gzip	× Non configurée explicitement	Pas de middleware de compression dans FastAPI ni dans la config Next.js — dépend du reverse proxy Render
Images WebP/AVIF	× Non applicable	Pas d'images binaires dans l'UI (icônes SVG uniquement) — ✅ de facto
CDN pour les assets statiques	× Absent	Next.js en mode standalone sans CDN configuré — Render ne fournit pas de CDN sur le plan Free
HTTP/2	⚠️ Dépend de Render	Render.com supporte HTTP/2 sur ses reverse proxies — activé automatiquement
Pagination sur les endpoints de liste	⚠️ Partielle	`GET /sessions` et `GET /users` retournent toutes les entrées sans pagination — à ajouter
Cache des résultats LLM	× Prévu non implémenté	Redis présent dans docker-compose mais non connecté au backend